

Review

Not peer-reviewed version

Advancing TinyML in IoT: A Holistic System-Level Perspective for Resource-Constrained AI

[Leandro Antonio Pazmiño Ortiz](#)*, [Ivonne Fernanda Maldonado Soliz](#), Vanessa Katherine Guevara Balarezo

Posted Date: 4 April 2025

doi: 10.20944/preprints202504.0344.v1

Keywords: Artificial Intelligence; IoT; Edge Computing; Resource-Constrained Devices; TinyML; Federated Learning; Model Compression; Security and Privacy



Preprints.org is a free multidisciplinary platform providing preprint service that is dedicated to making early versions of research outputs permanently available and citable. Preprints posted at Preprints.org appear in Web of Science, Crossref, Google Scholar, Scilit, Europe PMC.

Copyright: This open access article is published under a Creative Commons CC BY 4.0 license, which permit the free download, distribution, and reuse, provided that the author and preprint are cited in any reuse.

Review

Advancing TinyML in IoT: A Holistic System-Level Perspective for Resource-Constrained AI

Leandro Pazmiño Ortiz *, Ivonne Maldonado Soliz and Vanessa Guevara Balarezo

Escuela de Formación de Tecnólogos, Escuela Politécnica Nacional; leandro.pazmino@epn.edu.ec (L.P.);

vanessa.guevarav@epn.edu.ec (V.G.).

* Correspondence: leandro.pazmino@epn.edu.ec

Abstract: Resource-constrained devices, including low-power Internet of Things (IoT) nodes, microcontrollers, and edge computing platforms, have increasingly become the focal point for deploying on-device intelligence. By integrating artificial intelligence (AI) closer to data sources, these systems aim to achieve faster responses, reduce bandwidth usage, and preserve privacy. Nevertheless, implementing AI in limited hardware environments poses substantial challenges in terms of computation, energy efficiency, model complexity, and reliability. This paper provides a comprehensive review of state-of-the-art methodologies, examining how recent advances in model compression, TinyML frameworks, and federated learning paradigms are enabling AI in tightly constrained devices. We highlight both established and emergent techniques for optimizing resource usage while addressing security, privacy, and ethical concerns. We then illustrate opportunities in key application domains—such as healthcare, smart cities, agriculture, and environmental monitoring—where localized intelligence on resource-limited devices can have broad societal impact. By exploring architectural co-design strategies, algorithmic innovations, and pressing research gaps, this paper offers a roadmap for future investigations and industrial applications of AI in resource-constrained devices.

Keywords: Artificial Intelligence; IoT; Edge Computing; Resource-Constrained Devices; TinyML; Federated Learning; Model Compression; Security and Privacy

1. Introduction

The introduction should briefly place the study in a broad context and highlight why it is important. It should define the purpose of the work and its significance. The current state of the research field should be carefully reviewed and key publications cited. Please highlight controversial and diverging hypotheses when necessary. Finally, briefly mention the main aim of the work and highlight the principal conclusions. As far as possible, please keep the introduction comprehensible to scientists outside your particular field of research. References should be numbered in order of appearance and indicated by a numeral or numerals in square brackets—e.g., [1] or [2,3], or [4–6]. See the end of the document for further details on references. Artificial intelligence (AI) has seen extraordinary advances over the past decade, driven by improvements in deep neural network architectures, the availability of massive datasets, and the proliferation of specialized hardware accelerators [1]. As these technologies continue to mature, there is a growing interest in pushing AI capabilities beyond centralized data centers and cloud-based architectures to the very edge of the network, where data is initially generated and collected [2,3]. This shift in focus toward edge or embedded AI is driven by a set of converging demands, including the need to reduce inference latency for real-time applications, the desire to cut bandwidth costs by transmitting less data to the cloud, and the imperative to preserve user privacy through localized data processing [4]. Despite these strong motivations, implementing AI models on resource-constrained devices remains a formidable challenge. Traditional deep learning pipelines typically rely on large-scale computing

clusters, powered by GPUs or TPUs, which deliver the necessary memory and arithmetic throughput to train and infer from very large models [5].

In stark contrast, many IoT and embedded systems operate with only a few megabytes of memory, minimal floating-point capabilities, and extremely tight energy budgets, [6]. Consequently, the straightforward transfer of standard AI methods to edge devices inevitably encounters severe computational bottlenecks, prompting researchers and practitioners to explore alternative approaches [7]. Techniques such as model compression, which includes quantization, pruning, and knowledge distillation, have emerged as key enablers in shrinking neural network footprints for edge deployment. Simultaneously, frameworks like TensorFlow Lite for Microcontrollers and microTVM are specifically designed to streamline inference on microcontrollers and other resource-limited hardware [8]. Furthermore, federated learning (FL) has opened avenues for collaborative, decentralized model training, which not only conserves bandwidth but also enhances data privacy by keeping sensitive information local [9]. However, challenges persist, including the need to balance model accuracy with hardware limitations, ensure robust performance under real-time constraints, and maintain data security against sophisticated adversaries.

The overarching aim of this paper is to provide a thorough review of the challenges, opportunities, and emergent techniques that underpin the growing domain of AI on resource-constrained devices. By consolidating insights from recent research, we highlight how state-of-the-art methods address the issues of limited memory, limited compute capability, and strict energy budgets [10]. We also discuss how these solutions can be applied in various real-world scenarios, from wearable healthcare devices to environmental monitoring systems [11]. In exploring these application domains, we underscore the importance of ethical and sustainability considerations, recognizing that responsible AI deployment must factor in human well-being, data privacy, and ecological impact [12].

By systematically reviewing state-of-the-art methodologies, we provide a roadmap for improving AI deployment in edge computing and embedded systems.

2. Background and Motivation

2.1. The Rise of Edge and Embedded AI

The traditional cloud-centric model of deploying AI, wherein sensor data is streamed to powerful remote servers for processing, has been immensely successful in a variety of contexts. This approach leverages large-scale data storage and computing power to train and execute high-complexity models [2]. However, with the dramatic growth of the Internet of Things, projected to reach billions of devices across home automation, industrial settings, and urban infrastructure, the bandwidth and latency constraints of funneling all data to the cloud have become more pronounced [13]. High network costs, intermittent connectivity, and the pressing need for quick inferential decisions—especially in safety-critical applications like autonomous driving—have catalyzed interest in distributing intelligence to the network's periphery [14]. Edge or embedded AI addresses these issues by bringing computational processing closer to data sources, either at dedicated gateway nodes or directly on ultra-low-power IoT devices themselves [15].

This architectural shift not only reduces network overhead but can also allow for near-instantaneous decision-making. For instance, a wearable device that monitors cardiac signals can detect and alert users of arrhythmia without having to transmit continuous data streams to remote servers [16]. This local analysis yields immediate feedback to the end user and can conserve battery life by limiting wireless communications. Consequently, the integration of AI into the IoT ecosystem represents a natural evolution in how complex analytics are performed, enabling an array of new applications that demand low latency or operate in bandwidth-restricted settings [17,18].

2.2. Defining Resource-Constrained Devices

Resource-constrained devices are often microcontrollers or small single-board computers that exhibit stringent limitations in memory, compute capacity, and power supply [19]. These devices may only support a few megabytes of RAM and rely on low-frequency CPUs, making them vastly less capable than typical smartphones or personal computers [20]. Many of these devices operate intermittently on battery power or energy-harvesting sources such as solar or kinetic energy, adding further constraints to how often (and how intensively) AI tasks can be executed. Network connectivity also varies widely in such setups, with some devices operating in remote or isolated locations that only occasionally synchronize data [21].

Hardware diversity compounds these challenges. Unlike the relatively homogeneous environments of cloud data centers, embedded systems exist in a fragmented ecosystem of different instruction sets, memory hierarchies, and peripheral configurations [22]. This heterogeneity forces developers to consider platform-specific optimizations to ensure that AI pipelines run efficiently [23]. Moreover, tasks can differ dramatically between applications. A smart home device might only need to detect a wake word for voice control, while an industrial sensor might analyze vibration patterns in real time to predict machine anomalies [27]. In each scenario, limited hardware capabilities demand specialized techniques for model design, training, and deployment to ensure that performance metrics—such as inference time, accuracy, and energy usage—meet application requirements [25].

The practical implementation of TinyML applications is deeply intertwined with the capabilities and limitations of the underlying hardware, a landscape marked by significant resource constraints and platform heterogeneity. To concretely illustrate this operational environment, **Table 1** presents a selection of commonly used development boards prominent in the TinyML ecosystem. This table highlights the substantial diversity found across these platforms, detailing key specifications such as the Microcontroller Unit (MCU), CPU architecture, clock frequency, memory resources (Flash and SRAM), and physical form factor. By also listing typical applications for each board, the table underscores the varied hardware requirements driven by different edge AI tasks and provides context for the platform-specific optimization challenges inherent in the field.

Table 1. Examples of commonly used boards for TinyML projects.

Board	MCU	Main Features	Dimensions	Applications	References
Adafruit EdgeBadge	ATSAMD51J19	ARM Cortex-M4F, 120 MHz, 512 KB Flash, 192 KB SRAM	86.3 × 54.3 mm	Edge-based image recognition and processing using TinyML.	[26]
Arducam Pico4ML-BLE	RP2040	Dual-core ARM Cortex-M0+, 133 MHz, 2 MB Flash, 264 KB SRAM	51 × 21 mm	Data collection and lightweight image processing with TinyML models.	[27]
Arduino Nano 33 BLE Sense	nRF52840	Cortex-M4, 64 MHz, 1 MB Flash, 256 KB SRAM	45 × 18 mm	Voice recognition and motion sensing for IoT edge applications.	[28]
B-L475E-IOT01A Discovery kit	STM32L4	ARM Cortex-M4, 80 MHz, 1 MB Flash, 128 KB SRAM	61 × 89 × 9 mm	Direct cloud-connected IoT applications with TinyML for data pre-processing.	[29]
ESP32-S3-DevKitC	ESP32-S3-WROOM-1	32-bit Xtensa dual-core, 240 MHz, 8 MB Flash, 512 KB SRAM	N/A	Rapid prototyping of IoT systems with TinyML for real-time tasks.	[30]

Himax WE-I	HX6537-A	ARC 32-bit DSP, 400 MHz, 2 MB Flash, 2 MB SRAM	40 × 40 mm	High-performance image and voice sensing with ambient analysis for smart systems.	[31]
Jetson Nano	N/A	Quad-core ARM A57, 1.43 GHz, N/A Flash, 4 GB SRAM	70 × 45 mm	AI-driven robotics and computer vision with TinyML for autonomous systems.	[32]
Portenta H7	STM32H747	Cortex-M7 & Cortex-M4, 480 & 240 MHz, 16 MB NOR Flash, 8 MB SRAM	62 × 25 mm	Advanced computer vision, robotics, and lab setups for AI experimentation.	[33]
Raspberry Pi 4 Model B	BCM2711	Quad-core Cortex-A72, 1.5 GHz, N/A Flash, 2–8 GB SRAM	56.5 × 86.6 mm	Robotics and smart home automation with AI and TinyML at the edge.	[34]
Raspberry Pi Pico	RP2040	Dual-core ARM Cortex-M0+, up to 133 MHz, 2 MB Flash, 264 KB SRAM	51 × 21 mm	Wake-up word detection and lightweight TinyML edge processing.	[35]
Seeeduno XIAO	SAMD21G18	ARM Cortex-M0+, up to 48 MHz, 256 KB Flash, 32 KB SRAM	20 × 17.5 × 3.5 mm	Wearable device prototyping and real-time analytics with TinyML.	[36]
Sony Spresense	CXD5602	ARM Cortex-M4F (×6 cores), 156 MHz, 8 MB Flash, 1.5 MB SRAM	50 × 20.6 mm	Sensor data analysis and image processing for advanced AI systems.	[37]
SparkFun Edge	Apollo3	ARM Cortex-M4F, up to 96 MHz, 1 MB Flash, 384 KB SRAM	40.6 × 40.6 mm	Ultra-low power motion sensing for IoT edge applications with TinyML.	[38]
Syntiant TinyML	NDP101	Cortex-M0+, 48 MHz, 256 KB Flash, 32 KB SRAM	24 × 28 mm	Speech recognition and sensor interfacing for TinyML edge tasks.	[39]
Wio Terminal	ATSAMD51P19	ARM Cortex-M4F, 120 MHz, 4 MB Flash, 192 KB SRAM	72 × 57 mm	Remote control and monitoring systems with TinyML for low-latency tasks.	[40]

2.3. Significance of On-Device AI

Integrating AI directly into constrained devices offers several compelling benefits that align with modern technological and societal needs [41]. One advantage is **reduced inference latency**, as local processing eliminates the round-trip delay of sending data to a central server, which is critical in real-time systems like drones or industrial robots [42]. Another benefit is **improved privacy**, as raw data can remain on the device, thereby minimizing exposure to potential eavesdropping or data breaches. This aspect is particularly important in sectors handling sensitive information, such as healthcare and finance, where data protection regulations and ethical considerations are paramount [43].

Additionally, **bandwidth conservation** emerges as an important incentive for on-device processing. As the volume of IoT data grows, transmitting this data continuously to a remote server can strain networks, particularly in rural or remote settings where connectivity is sporadic or expensive [44]. By performing key inferences locally, the system drastically reduces the volume of data needed to be sent. In scenarios such as environment monitoring or agricultural sensing, local AI can differentiate between normal and exceptional conditions, ensuring that only relevant data

triggers an alert or a database update [45]. Beyond these technical motivations, on-device AI also paves the way for large-scale, **distributed intelligence** architectures, where billions of devices can each contribute to localized analysis, collectively forming a robust, decentralized network of computing resources [46].

3. Key Challenges in AI for Constrained Devices

3.1. Computational and Memory Limitations

One of the most significant challenges in edge AI stems from the inherently limited computational resources available on these devices. Many microcontrollers lack hardware floating-point units or are restricted to a handful of specialized accelerators [47]. Standard deep neural networks, which can contain millions or even billions of parameters, are highly impractical in this setting due to the sheer size of their model weights and the complexity of their mathematical operations [42]. Memory constraints further exacerbate the problem, as loading large models into main memory can easily exceed the device's capacity. Even storing intermediate activations during inference can become a hurdle if the model is not carefully optimized [14].

These limitations place a ceiling on the complexity of algorithms that can be feasibly deployed on constrained hardware. For example, a device with a few hundred kilobytes of RAM might only be able to accommodate simple feedforward networks or highly compressed convolutional architectures [19]. If the model demands recurrent computations or multi-branch topologies, the overhead for storing hidden states and intermediate layer outputs might overwhelm the device's resources [48]. Consequently, edge-focused AI has turned to novel compression and optimization techniques to ensure that the final compiled model fits within the device's memory envelope while still achieving acceptable accuracy levels [21].

3.2. Energy Efficiency and Thermal Management

Energy efficiency is paramount in many edge deployments, especially in battery-powered or energy-harvesting scenarios. Executing computationally heavy AI tasks often leads to rapid battery drain, which not only shortens device lifespan but can also introduce thermal management challenges if components overheat [49]. The interplay between computation and power usage is complex: while simpler models require fewer multiply-accumulate operations, they might necessitate more frequent inference cycles to achieve real-time responsiveness. Designers must carefully weigh these trade-offs when deciding on an AI architecture or inference schedule [50].

Furthermore, certain embedded use cases demand continuous operation to provide always-on monitoring. This requirement can conflict with the desire to conserve power through extended sleep modes [51]. Hence, techniques such as dynamic voltage and frequency scaling, scheduling AI workloads to periods of ample power availability, and advanced power gating for idle subsystems are being explored [52]. These solutions aim to strike a balance between responsiveness and energy consumption, ensuring that the device can sustain its tasks for the desired operational duration. The need for efficient thermal design is also non-trivial, as high-performance AI tasks can generate heat in tightly enclosed form factors, prompting consideration of lightweight heat sinks or self-regulating workloads to avoid temperature-induced failures [53].

3.3. Real-Time Constraints

Real-time applications—such as robotics, autonomous systems, and certain industrial processes—add another layer of complexity to AI deployment on constrained devices. In these scenarios, decisions must be made within strict timing windows to ensure safe and effective operation [54]. Consider a robotic arm in a smart factory that must adapt its movements based on vision-based AI inferences: if the computations are delayed beyond a few milliseconds, the entire control loop could become unstable, leading to errors or safety issues. Balancing the intricacy of AI models, which

often correlates with their predictive accuracy, against the system's real-time requirements is a key challenge in edge AI [55].

Techniques to address this challenge typically involve model simplification or designing specialized hardware accelerators that can run inference quickly. Real-time operating systems (RTOS) may also be employed to guarantee scheduling priorities for AI tasks, ensuring that they preempt lower-priority processes. In some instances, partial offloading strategies are used, where less time-critical workloads are sent to nearby edge servers or the cloud when feasible, while mission-critical tasks stay local [56]. Nevertheless, the margins for error are slim, and ensuring deterministic response times frequently pushes developers toward rigorous profiling and optimization at every stage of the pipeline, from data acquisition to final output [47].

3.4. Data Security and Privacy

Although on-device inference can enhance privacy by limiting data exposure, many security vulnerabilities still exist. Edge devices deployed "in the wild" are physically accessible and can be tampered with by adversaries who might extract sensitive information from stored models or training parameters [41]. Lightweight encryption and secure boot processes become essential for protecting both firmware integrity and any locally stored data. In more advanced scenarios, secure enclaves or hardware-based trusted execution environments (TEEs) can isolate sensitive computations from other system components [45].

Moreover, privacy is not solely a matter of data encryption. Complex, intelligent systems can inadvertently leak information through side channels or by revealing model outputs [57]. Federated learning, for instance, transmits model updates or gradients rather than raw data, but these gradients can sometimes be reversed or manipulated to expose sensitive details about the local dataset [58]. As a result, implementing techniques like differential privacy or homomorphic encryption in resource-constrained settings becomes an active area of research. Balancing robust security with minimal overhead is challenging, as cryptographic operations themselves can be computationally expensive, consuming both energy and processing cycles that are already at a premium [59].

3.5. Model Generalization vs. Specialization

Another persistent dilemma is deciding between highly specialized models tailored to a single task and more generalized models that can handle multiple tasks or adapt to new conditions. Specialized models tend to be smaller and more efficient, since they incorporate only the parameters needed for a specific domain [12]. This approach can be ideal for devices that serve well-defined functions, such as a vibration sensor used exclusively for bearing fault detection. However, if the use case evolves—say, the sensor must also detect temperature anomalies—then a specialized model might not transfer well to the new requirement [60].

On the other hand, more generalized architectures can support multiple tasks but often at the cost of increased parameter counts and complexity, which can be prohibitive for constrained environments [61]. Transfer learning or incremental learning techniques that adapt a pre-trained model to new tasks on-device offer potential workarounds, but they introduce additional inference or fine-tuning overhead that might exceed the device's resource budget [21]. Researchers continue to explore ways to construct models that are flexible yet compact, possibly through dynamic neural networks that can switch off unused modules or layers to meet different objectives [62].

3.6. Reliability and Fault Tolerance

Edge devices may operate in harsh or unpredictable conditions, such as extreme temperatures, vibrations, or exposure to moisture, which can degrade hardware components and sensors over time [63]. Ensuring consistent AI performance in these environments demands robust hardware design, but it also relies on fault-tolerant software that can handle partial failures gracefully. For instance, sensor readings may become noisy or intermittent, yet the AI system must still produce reliable

inferences [12]. Traditional machine learning models trained on clean, consistent datasets may falter in such real-world conditions.

In response, developers employ redundancy strategies, sensor fusion, and error-correcting techniques to mitigate hardware-induced variability [64]. At the software layer, models can be trained with augmented data that simulate potential noise or missing channels, improving resilience during inference [65]. Another approach is to integrate continual learning methods that periodically retrain or fine-tune models in situ, helping the system adapt to evolving environmental conditions or device aging [9]. These strategies, while effective, add additional layers of complexity that must be balanced against already-constrained resources.

4. Emerging Solutions and State-of-the-Art Approaches

4.1. Model Compression Techniques

Model compression stands at the forefront of enabling AI in resource-constrained environments by reducing the memory footprint and computational demands of neural networks [66]. Among these techniques, **quantization** is a widely adopted method that converts floating-point weights and activations to lower bit representations—commonly 8-bit integers—to diminish the size of the model and accelerate inference [67]. Advanced quantization approaches push precision even lower, exploring 4-bit or 1-bit representations, though such aggressive strategies can degrade accuracy if not carefully calibrated and retrained [45]. Tools like TensorFlow Lite provide built-in utilities for post-training quantization, which can drastically cut the size of pre-trained models with minimal performance loss [68].

Pruning and sparsity target the removal or zeroing out of redundant weights or channels within a network to reduce computation. Structured pruning, for instance, systematically eliminates entire filters in convolutional layers, leading to more regular sparsity patterns that can be more efficiently accelerated by specialized hardware [69]. Combined with quantization, pruning can reduce model size by an order of magnitude or more, making it feasible to deploy deep neural networks on microcontrollers with very limited memory resources. **Knowledge distillation** offers another powerful technique, in which a smaller “student” network learns to emulate the outputs of a larger “teacher” model, effectively capturing the teacher’s learned representations while operating on a fraction of the parameters [70]. Often, these techniques are combined in a holistic approach to maximize memory savings, striking a fine balance between a model’s resource footprint and inference accuracy [12].

Among the various model compression techniques vital for enabling AI on constrained hardware, quantization and pruning represent two foundational approaches, often used in conjunction. While both aim to reduce model size and computational cost, they operate via distinct mechanisms and entail different trade-offs regarding accuracy, hardware compatibility, and implementation complexity. To clarify their respective strengths and weaknesses, Table 2 offers a critical analysis comparing these methods across several key dimensions. This includes their typical impact on memory footprint, suitability for hardware acceleration, potential effects on model accuracy, deployment complexity, and achievable real-world speedups, thereby highlighting the essential considerations involved in selecting and applying these techniques for TinyML applications.

Table 2. Critical Analysis: Quantization vs. Pruning: Accuracy vs. Speed Trade-Offs.

Dimension	Quantization	Pruning	References
Memory Footprint	High reduction (e.g., 4–8×) by reducing numerical precision.	Moderate–high reduction by removing entire weights/channels.	[67,68]
Hardware Acceleration	Typically straightforward to accelerate with integer ops.	Dependent on kernel support for sparse operations.	[69]

Accuracy Impact	Generally small if calibrated or fine-tuned; can degrade if precision is too aggressive.	Can be significant if excessively pruned without careful retraining.	[70]
Deployment Complexity	Often simpler (e.g., post-training quantization or quant-aware training).	Requires re-training or iterative search for optimum sparsity.	[71]
Real-World Speedups	Typically consistent on integer-friendly hardware; improved throughput/latency.	Highly dependent on structured vs. unstructured pruning and hardware’s ability to exploit sparsity.	[69]

4.2. TinyML Frameworks

The growing popularity of **TinyML** has spawned specialized frameworks and toolchains designed to streamline the deployment of machine learning models on resource-constrained devices. One of the most prominent examples is **TensorFlow Lite for Microcontrollers**, a pared-down inference engine that can execute neural network operations in a memory footprint on the order of tens of kilobytes [8]. By trimming unnecessary runtime features and relying on carefully optimized kernels, TensorFlow Lite for Microcontrollers can run on various microcontroller platforms, including ARM Cortex-M series CPUs and ESP32 boards. Another framework, **microTVM**, extends the TVM compiler infrastructure to facilitate automated code generation and hardware-specific optimizations, enabling developers to explore different back-end configurations [71].

Additionally, **CMSIS-NN** from ARM enhances performance of neural network computations by providing hand-optimized kernels for common operations like convolution and activation functions. These frameworks typically require developers to conduct model training offline on a more capable system, after which the models undergo optimization steps like quantization or weight compression [72]. The resulting executables can then be flashed onto the embedded device. The synergy between these specialized libraries and carefully pruned or quantized models allows even limited systems to achieve meaningful AI functionality in applications like keyword spotting, anomaly detection, and basic computer vision tasks.

Successfully deploying optimized machine learning models onto microcontrollers necessitates more than just model compression; it requires specialized software frameworks designed to manage inference within severe resource limitations. These frameworks provide runtime engines, optimized mathematical kernels, and often tools for model conversion and code generation tailored to embedded environments. To provide developers with an overview of the available toolchains, **Table 3** presents a comparative analysis of popular TinyML frameworks, such as TensorFlow Lite for Microcontrollers, microTVM, and Edge Impulse. The table summarizes their core features, discusses their primary advantages (Pros) and disadvantages (Cons), offering insights into their suitability for different development workflows, target hardware platforms, and optimization requirements.

Table 3. Benchmarking data: popular TinyML framework comparison.

Framework	Core Features	Pros	Cons	References
TFLite Micro	Optimized for 8-bit quant. Minimal runtime & memory usage.	Mature ecosystem. Large user community & docs. Wide device support.	May require manual tuning for advanced tasks. C++ heavy.	[8]
microTVM	Compiler-level optimizations. Automated code generation.	Flexible & hardware-agnostic. Supports multiple back-ends.	Steep learning curve. Some features in active dev.	[71]

Edge Impulse	Cloud-based pipeline. AutoML & sensor data ingestion.	Rapid prototyping. No/low-code approach. Integrated IDE.	Cloud dependency. Possible vendor lock-in.	[73]
CMSIS-NN	Hand-optimized kernels for ARM MCUs.	Very fast conv & activation routines. Directly integrates with MCU ecosystem.	Primarily ARM-focused. Less feature-rich than TFLM/microTVM.	[72]

4.3. Hardware Accelerators and Architectures

Although general-purpose microcontrollers have improved steadily, some deployments demand specialized hardware to meet strict performance and power goals. **ASICs (Application-Specific Integrated Circuits)** can be optimized around specific neural network layers or dataflow patterns to reduce overhead and deliver higher throughput per watt. By tailoring the memory hierarchy and arithmetic units to AI workloads, designers can minimize data movement, a major source of energy consumption. This leads to significant improvements in efficiency, although development costs are non-trivial [53].

FPGA-based accelerators offer a reconfigurable alternative, enabling hardware-level parallelism without sacrificing programmability. Developers can adapt an FPGA’s logic blocks and interconnect to accelerate convolutional layers, matrix multiplications, or other computational bottlenecks. This adaptability is particularly useful for rapidly evolving AI workloads, but can pose challenges in terms of design complexity and timing closure. A more radical direction involves **in-memory computing**—placing computation logic directly within memory arrays to drastically cut data transfer overhead [74].

Many microcontrollers lack dedicated MAC (multiply-accumulate) units or hardware accelerators, driving research into custom ASICs, FPGAs, and near-memory computing for TinyML workloads. Hardware-aware neural architecture search (NAS) can produce specialized topologies that conform to memory or MAC-operation constraints (e.g., **TinyTNAS** for time series classification). Some designs adopt data-channel extension (DEX) to better utilize the limited parallelism in tiny AI accelerators, improving throughput [11]. RISC-V-based heterogeneous architectures with built-in co-processors for neural nets have also been proposed [75].

Table 4 presents a comparison of various TinyML hardware platforms, highlighting their computing power, memory constraints, and suitability for machine learning applications at the edge.

Table 4. Comparison of TinyML-Compatible Hardware Platforms.

Device	Architecture	Frequency (MHz)	RAM (KB/MB)	TinyML Capability	References
ESP32-S3	Xtensa LX7 (dual-core)	240	512 KB	Medium (basic audio and simple inferences)	[30]
Raspberry Pi Pico	ARM Cortex-M0+	133	264 KB	Low (only very lightweight models)	[35]
Arduino Nano 33 BLE Sense	ARM Cortex-M4	64	256 KB	High (sensor integration and embedded ML)	[28]

Jetson Nano	ARM Cortex-A57	1420	4 GB	Very High (deep learning and computer vision)	[32]
-------------	----------------	------	------	---	------

4.4. Federated Learning and Edge AI

Federated learning (FL) has emerged as a powerful technique for collaborative model training across distributed, resource-constrained devices without requiring raw data to leave each node [12]. In a typical FL setup, each device locally trains a model using its own dataset and periodically sends only model updates or gradients to a central server, which aggregates and updates a global model [76]. This architecture can significantly reduce the bandwidth burden associated with traditional centralized training and helps preserve privacy, as personal data rarely traverses the network in raw form [9]. However, many challenges remain, including dealing with heterogeneous hardware, unbalanced data distributions, and asynchronous update cycles.

Beyond federated learning, the broader concept of **edge AI** seeks to partition intelligence tasks among end devices, edge servers, and the cloud in a hierarchical fashion [77]. Low-latency tasks or privacy-sensitive computations are handled locally, while more computationally demanding operations can be offloaded to powerful edge servers when necessary [57]. This flexible approach acknowledges the heterogeneity of device capabilities within an IoT ecosystem and allows for more complex collaborative scenarios [58]. For instance, certain data might be partially processed on a microcontroller and then encrypted and forwarded to an edge server for refined analysis [60]. The design of such distributed architectures involves careful orchestration of resource scheduling, data caching, and robust communication protocols [9].

4.5. Security Mechanisms and Lightweight Encryption

Securing AI pipelines in resource-limited settings is an integral component of deploying edge intelligence at scale. While the focus is often on performance optimization, insufficient attention to security can expose critical infrastructures to malicious threats [55]. **Lightweight cryptography** protocols, which are specifically adapted to devices with small footprints, have gained traction [67]. These protocols strike a balance between algorithmic complexity and security guarantees, ensuring that data remains encrypted during transmission and at rest with minimal overhead. As an example, elliptic-curve cryptography (ECC) variants can offer strong encryption with relatively smaller key sizes than RSA-based approaches, thus reducing computational cost [68].

Beyond encryption, **trusted execution environments (TEEs)** such as ARM TrustZone or Intel SGX enable the creation of secure enclaves that isolate sensitive computations from the rest of the system [69]. In the context of AI, TEEs can store and execute model parameters securely, preventing adversarial access. Additional layers of defense, like secure bootloaders, can ensure that only verified firmware is executed on the device, thwarting attempts to run malicious code. Implementing such measures requires a comprehensive approach that covers hardware, software, and network layers, all while preserving the fundamental constraints of power, memory, and compute capacity that characterize edge devices [45].

4.6. Software and System Optimizations

System-level optimizations that transcend individual models or frameworks also play a pivotal role. Embedded or real-time operating systems must provide efficient task scheduling to ensure that AI inference does not monopolize the CPU or block critical sensing routines [70]. Approaches like event-driven task scheduling or preemptive multitasking can coordinate low-priority background tasks with higher-priority inference jobs [16]. **Dynamic power management**—where system components enter low-power states when idle—further helps to conserve energy in always-on monitoring applications. Moreover, techniques like sensor fusion can reduce redundant processing

steps by combining complementary data streams, potentially lowering the overhead required for complex AI inference [50].

Another emerging theme is the use of containerization and virtualization in resource-limited contexts, although these concepts remain largely uncharted territory in extremely constrained microcontroller environments [12]. Lightweight container solutions may one day permit multiple AI applications to coexist on a single embedded platform, each securely isolated to maintain reliability and data confidentiality. However, such advanced features typically come with non-trivial memory and CPU overhead, illustrating the perennial trade-off between functionality and resource usage [75]. Moving forward, bridging the gap between full-fledged operating systems and bare-metal solutions is likely to inspire new forms of specialized runtime environments optimized for AI at the edge [21].

5. Opportunities and Cross-Cutting Themes

5.1. Healthcare and Wearables

Healthcare stands as one of the most promising domains where resource-constrained AI can have a transformative impact [16]. **Wearable devices**, such as smartwatches, fitness bands, or specialized medical sensors, are increasingly incorporating machine learning capabilities to track vital signs, detect abnormalities, and offer personalized health recommendations [12]. By performing AI inference locally, these wearables can protect sensitive health information from being routinely transmitted to the cloud, significantly reducing privacy risks and potential regulatory hurdles [73]. For patients with chronic conditions like diabetes or cardiac arrhythmia, continuous on-device monitoring can detect early warning signs and trigger timely interventions.

Nevertheless, wearables must maintain battery life over extended periods to remain unobtrusive and user-friendly. Achieving this goal requires efficient AI architectures that can run quickly and minimize power drain [41]. Additionally, user comfort and adoption hinge on the devices' physical form factor, which limits the potential for large heat sinks or bulky batteries. Innovations in model compression and hardware-software co-design can help address these constraints, as can synergy with other medical IoT components like smart patches or implantables [78]. Over time, these converging solutions may result in integrated healthcare ecosystems where personalized diagnostics and treatment plans are partly executed on ultra-low-power devices worn by patients in real-world settings [21].

5.2. Smart Cities and Infrastructure

City planners worldwide are increasingly relying on sensor networks to monitor and manage critical infrastructure, spanning traffic flow, waste management, air quality, and energy distribution [74]. Implementing **AI at the edge** within these urban ecosystems enables real-time decision-making, such as rerouting traffic after an accident or identifying energy waste in smart grids [3]. By embedding intelligence in streetlights, traffic signals, and other municipal assets, cities can adapt dynamically to evolving conditions, optimizing resource allocation and reducing operational costs. Furthermore, localized AI processing reduces the bandwidth needed for collecting vast volumes of raw sensor data, thus making large-scale sensor deployments more feasible [77].

Yet, the sheer scale of smart city infrastructures introduces complexities. Citywide sensor networks comprise thousands or even millions of devices with highly varied hardware profiles and communication methods [66]. Managing firmware updates, security patches, and consistent model deployment across such a heterogeneous landscape becomes a monumental undertaking [52]. Federated learning approaches might alleviate some challenges by enabling distributed model training, but guaranteeing reliability and security at such a scale also necessitates robust device identification, tamper-resistant hardware, and flexible policy enforcement [79]. If successfully implemented, resource-constrained AI in smart city environments can provide actionable insights, streamline municipal services, and promote sustainability [45].

5.3. Agriculture and Environmental Monitoring

Another domain ripe for resource-constrained AI is **smart agriculture**, where low-cost sensors and drones can collect data on soil moisture, temperature, crop health, and pest infestations. By analyzing these signals in situ, farmers can make data-driven decisions about irrigation schedules, pesticide use, and harvesting times, improving yields and minimizing waste [80]. Importantly, many agricultural lands are remote, lacking stable internet connectivity, making local AI inference essential. Processing data locally can drastically reduce the volume of transmissions—limited to event triggers or summary statistics—thus preserving battery life for devices powered by solar panels or other off-grid sources [81].

Similarly, **environmental monitoring** efforts depend on scattered sensors tracking air, water, or wildlife conditions across large geographic areas, including wilderness or marine habitats. Resource-constrained devices capable of on-site inference can detect pollution spikes, illegal deforestation, or changes in animal populations, alerting authorities only when anomalies occur [82]. This approach is more sustainable than continuously streaming high-resolution data, which may be difficult in areas with limited infrastructure [21]. Additionally, distributed sensor networks can offer robust resilience if certain nodes go offline, since intelligence is not solely centralized. By empowering local AI capabilities, environmental monitoring systems foster more immediate and effective conservation interventions while operating under constrained power and connectivity conditions [65].

5.4. Industry 4.0 and Predictive Maintenance

The concept of **Industry 4.0** emphasizes automation, data exchange, and interconnectivity in manufacturing systems. Factories increasingly employ sensors for condition-based monitoring, quality control, and predictive maintenance, aiming to predict machine failures before they occur [81]. Embedding AI into these sensors or edge gateways can enable real-time anomaly detection, reducing unplanned downtime and increasing overall equipment effectiveness [74]. Instead of collecting vast amounts of raw vibration or acoustic signals from each machine, local AI can parse relevant features, identify outliers, and send only crucial insights to a centralized platform, thus lowering bandwidth demands [72].

In practice, these environments are often harsh, with vibration, dust, and potential interference from other machinery. Resource-constrained AI hardware must be ruggedized and shielded against electromagnetic disturbances [75]. Moreover, the diversity of industrial protocols and hardware calls for flexible solutions capable of interfacing with legacy systems while delivering advanced AI. Intermittent connectivity or strict real-time control loops amplify the challenges. As with other domains, solutions for Industry 4.0 are gravitating toward compressed and efficient models, specialized accelerators, and co-design strategies that integrate the entire data pipeline, from sensor to analytics, under a unified optimization framework [10].

5.5. Ethical and Sustainability Considerations

While technical progress in embedded AI is crucial, **ethical and sustainability** dimensions cannot be overlooked. Data bias and representativeness pose major concerns, particularly when training or refining models on localized datasets that may not capture the diversity of real-world conditions [83]. This risk is aggravated if multiple edge devices gather skewed or incomplete data, inadvertently reinforcing biases in collaborative systems. Careful curation, federated data sharing among diverse demographics, and ongoing validation of model outputs can mitigate this problem, though each step increases complexity [41].

Privacy also emerges as both a motivator and a challenge. On-device analysis does lower the risk of mass data collection, but it does not inherently solve transparency issues around what is being inferred and how decisions are made. Regulators and consumers alike demand clarity on how AI-based devices handle personal or environmental data. Furthermore, the surge in IoT device manufacturing raises environmental concerns, as large-scale hardware production consumes

materials and energy [78]. Proponents of edge AI argue that locally processed data can significantly lower global data center energy usage; however, the net ecological impact depends on device lifecycles, e-waste handling, and responsible recycling [84]. Thus, a holistic perspective—integrating the entire product chain—is key to ensuring that AI in constrained devices yields net benefits to society and the planet [51].

6. Future Directions

As AI deployment in resource-constrained settings accelerates, a plethora of research opportunities and practical challenges lie ahead. This section outlines several directions that hold promise for advancing the field.

6.1. *Dynamic Model Adaptation and On-Device Training*

The majority of edge AI solutions rely on static, pre-trained models that seldom adapt after deployment. However, real-world conditions are rarely static; input distributions can drift over time as environments change or user behaviors shift [58]. Therefore, techniques for **dynamic model adaptation**, such as runtime pruning or partial retraining, are expected to gain traction. By allowing models to adjust their parameters or structure based on real-time feedback, devices can maintain higher accuracy without frequent server communications [85]. The advent of incremental and transfer learning for microcontrollers remains an active research frontier, as training steps themselves can be computationally taxing. Nevertheless, approaches like federated or split learning could enable incremental updates, provided that algorithms are carefully optimized for local hardware constraints [86].

6.2. *Co-Design of Hardware and Algorithms*

Hardware-software co-design involves developing AI algorithms and hardware architectures synergistically, ensuring that each is optimized for the other [55]. Rather than designing a generic neural network and attempting to retrofit it for an embedded processor, co-design encourages selecting or inventing neural architectures that map efficiently onto specialized hardware blocks. Techniques like memory-centric computing or near-sensor intelligence can drastically cut energy usage by reducing data transfers [21]. For instance, certain convolutional neural network (CNN) layers can be implemented as streaming pipelines directly connected to sensor inputs, circumventing the need for large intermediate buffers [75]. As the AI ecosystem becomes increasingly heterogeneous, we anticipate novel instruction sets and domain-specific accelerators designed in tandem with lightweight neural models to catalyze breakthroughs in efficiency [53].

6.3. *Advanced Security and Trust Mechanisms*

Securing AI workflows on constrained devices calls for **continuous innovation** in cryptography, secure enclaves, and tamper-evident hardware. One emerging trend is the exploration of **homomorphic encryption**, which allows computations to be performed on encrypted data without exposing raw information [45]. Although computationally heavy for large-scale networks, scaled-down variants of homomorphic encryption might become more feasible as specialized hardware accelerators evolve. Another area of interest is secure multiparty computation (MPC), which enables multiple parties to jointly compute functions over their data while keeping inputs private [55]. Applied to federated learning, MPC can further mitigate privacy risks when aggregating updates across untrusted devices. Ensuring that these security measures remain lightweight enough for microcontrollers remains a central challenge [10].

6.4. *Privacy-Preserving Learning at Scale*

Federated learning has demonstrated the feasibility of distributed AI, yet scaling to millions or billions of edge devices can introduce performance bottlenecks, fairness concerns, and vulnerability

to malicious updates [87]. Researchers are investigating novel aggregation algorithms, incentive mechanisms, and robust outlier detection strategies to sustain large-scale, federated deployments [58]. Additionally, **differential privacy** frameworks aim to provide formal guarantees that individual data points cannot be re-inferred from aggregated model parameters, an especially relevant consideration for sensitive medical or personal data. Combining such techniques with compression and resource-aware scheduling is essential to ensure that large-scale federated learning remains computationally viable on highly constrained devices [12].

6.5. Resilience and Reliability for Real-World Environments

Edge devices may operate in harsh or unpredictable conditions, such as extreme temperatures, vibrations, or exposure to moisture, which can degrade hardware components and sensors over time [14]. Ensuring consistent AI performance in these environments demands robust hardware design, but it also relies on fault-tolerant software that can handle partial failures gracefully. For instance, sensor readings may become noisy or intermittent, yet the AI system must still produce reliable inferences [12]. Traditional machine learning models trained on clean, consistent datasets may falter in such real-world conditions. In response, developers employ redundancy strategies, sensor fusion, and error-correcting techniques to mitigate hardware-induced variability [58].

At the software layer, models can be trained with augmented data that simulate potential noise or missing channels, improving resilience during inference. Another approach is to integrate continual learning methods that periodically retrain or fine-tune models in situ, helping the system adapt to evolving environmental conditions or device aging [9]. These strategies, while effective, add additional layers of complexity that must be balanced against already-constrained resources [56].

6.6. Benchmarking and Standardization

As the field grows, the lack of universally accepted **benchmarks and metrics** for AI on constrained devices presents hurdles to reproducibility and fair evaluation [88]. While standard image classification or object detection benchmarks exist for large-scale networks, they do not necessarily reflect the unique constraints of TinyML contexts [25]. Moreover, measuring energy consumption, memory usage, and inference latency in a consistent manner can be challenging, given the diversity of hardware platforms. Initiatives like MLPerf Tiny, which focus on embedded inference, are steps in the right direction [21]. Further standardization efforts would promote transparent comparisons of compression methods, frameworks, and hardware designs, ultimately accelerating the pace of innovation in the ecosystem [10].

7. Opportunities and Future Outlook

The rapid expansion of the Internet of Things and the demand for real-time, privacy-preserving analytics have fueled the drive to incorporate **artificial intelligence into resource-constrained devices**. By migrating inference and, in some cases, training from centralized cloud servers to embedded microcontrollers, this paradigm promises to reduce latency, conserve bandwidth, and protect sensitive data [45]. However, these benefits come at the cost of formidable **technical challenges**, as typical deep learning pipelines must be significantly re-engineered to fit within tight memory, compute, and power budgets. Researchers and practitioners have responded with a surge of innovations, including model compression (quantization, pruning, knowledge distillation), TinyML frameworks, hardware accelerators, federated learning, and advanced security mechanisms [10].

This paper has explored the **key challenges**—spanning computational constraints, energy efficiency, real-time requirements, security, privacy, and reliability—that confront AI deployment in edge devices. We have then reviewed **state-of-the-art solutions**, highlighting how the synergy of specialized architectures, optimized software stacks, and collaborative learning protocols makes on-device intelligence increasingly viable [55]. Moreover, by examining **opportunities** in healthcare,

smart cities, agriculture, and Industry 4.0, we revealed the breadth of potential applications poised to benefit from localized AI inference. In parallel, we emphasized that **ethical and sustainability** considerations remain central to responsible AI adoption, particularly as billions of edge devices enter our homes, streets, and workplaces [66].

Looking ahead, **future directions** such as dynamic model adaptation, co-design of hardware and algorithms, advanced security mechanisms, large-scale privacy-preserving learning, and rigorous benchmarking offer fertile ground for continued progress [21]. The development of robust, low-power, and secure AI solutions capable of thriving under real-world conditions will not only require breakthroughs in algorithms and hardware but also interdisciplinary collaboration among data scientists, embedded systems engineers, ethicists, and policymakers [41]. Overcoming these challenges will pave the way for a new generation of pervasive, intelligent devices that seamlessly integrate into everyday life while preserving privacy, efficiency, and sustainability. In doing so, edge AI stands to transform industries and societies, enabling us to harness the power of machine intelligence wherever data is generated, even in the most constrained environments [10].

8. Key Insights Box

Key Insights for TinyML Researchers

1. Embrace System-Level Co-Design: AI compression alone is insufficient; synergy between algorithms, firmware, and hardware is critical for meaningful speedups and reliability [53,55].

2. Benchmark Under Real Constraints: True performance cannot be assessed without measuring energy consumption, latency, and memory usage against real application scenarios, especially under harsh or unpredictable conditions [10,21].

3. Layered Security is Essential: On-device inference helps preserve privacy, but resource-starved devices also require robust encryption, secure enclaves, and trustworthy firmware pipelines to combat tampering or data leakage [45,55].

4. Adaptation and Specialization: Balancing specialized models (extreme efficiency) with adaptive or multi-task approaches (flexibility) remains an open research question in resource-constrained AI [12,62].

5. Ethical and Sustainable Deployments: Large-scale IoT expansions demand addressing fairness, bias, environmental impact, and responsible data management, ensuring TinyML's long-term societal benefits [78].

10. Conclusions

The rapid expansion of the Internet of Things and the demand for real-time, privacy-preserving analytics have fueled the drive to incorporate on-device **artificial intelligence** in resource-constrained devices. By migrating inference—and, in some cases, training—from centralized cloud servers to embedded microcontrollers, this paradigm holds the promise of reducing latency, conserving bandwidth, and protecting sensitive data. However, these benefits entail overcoming a wide array of technical hurdles, from managing stringent memory and compute limits to securing physically exposed hardware and meeting real-time performance.

We reviewed the key challenges encountered in deploying AI on constrained hardware, focusing on computational limits, energy efficiency, real-time requirements, security, privacy, and reliability. We further examined the tools and techniques essential to address these constraints, including cutting-edge model compression, specialized TinyML frameworks, hardware accelerators, federated learning mechanisms, and advanced security approaches. Concrete application scenarios in healthcare, smart cities, agriculture, and industry highlight the transformative potential of embedded AI while also illustrating the unique demands placed upon these devices.

Ultimately, the future of TinyML depends not only on individual breakthroughs in compression or specialized hardware but on **system-level integration**, wherein every layer—from neural

architecture design to runtime scheduling and cryptographic defense—is tuned for synergy within extreme resource limits. The success of such holistic solutions will facilitate a new era of distributed AI that is both efficient and ethically grounded, extending sophisticated analytics to the remotest corners of our increasingly connected world.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature* **2015**, 521, 436–444.
2. M. Satyanarayanan. The emergence of edge computing. *Computer* **2017**, 50, 30–39.
3. R. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu. Edge computing: Vision and challenges. *IEEE Internet of Things J.* **2016**, 3, 637–646.
4. S. Li, L. Da Xu, and S. Zhao. The internet of things: A survey. *Inf. Syst. Front.* **2015**, 17, 243–259.
5. K. Bayoudh. “A survey of multimodal hybrid deep learning for computer vision: Architectures, applications, trends, and challenges”. *Information Fusion*, 105, 102217. <https://doi.org/10.1016/j.inffus.2023.102217>
6. S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2016.
7. D. Silver et al.. Mastering the game of Go with deep neural networks and tree search. *Nature* **2016**, 529, 484–489.
8. P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O'Reilly Media, 2019.
9. Q. Yang, Y. Liu, T. Chen, and Y. Tong. Federated machine learning: Concept and applications. *ACM Trans. Intell. Syst. Technol.* **2019**, 10, 1–19.
10. A. Elhanashi, P. Dini, S. Saponara, and Q. Zheng. Advancements in TinyML: Applications, Limitations, and Impact on IoT Devices. *Electronics*, 2024. [Online]. <https://doi.org/10.3390/electronics13173562>
11. L. D. Xu, W. He, and S. Li. Internet of Things in industries: A survey. *IEEE Trans. Ind. Informat.* **2014**, 10, 2233–2243.
12. N. Alajlan and D. M. Ibrahim. TinyML: Adopting tiny machine learning in smart cities. *Journal of Autonomous Intelligence*, 2024. [Online]. Available: <https://doi.org/10.32629/jai.v7i4.1186>
13. M. Giordano et al.. Design and Performance Evaluation of an Ultralow-Power Smart IoT Device With Embedded TinyML for Asset Activity Monitoring. *IEEE Transactions on Instrumentation and Measurement*, NaN. [Online]. Available: <https://doi.org/10.1109/tim.2022.3165816>
14. E. Nurvitadhi et al.. Can FPGAs beat GPUs in accelerating next-generation deep neural networks?. in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2017, pp. 5–14.
15. H. Han and J. Siebert. TinyML: A Systematic Review and Synthesis of Existing Research. in *Digital Signal Processing and Signal Processing Education Workshop*, 2022. [Online]. Available: <https://doi.org/10.1109/ICAIIC54071.2022.9722636>
16. M. Johnvictor, P. M., S. N. Prem, and T. Vs. TinyML-Based Lightweight AI Healthcare Mobile Chatbot Deployment. *Journal of Multidisciplinary Healthcare*, 2024. [Online]. Available: <https://doi.org/10.2147/JMDH.S483247>
17. N. H. Motlagh, M. Bagaa, and T. Taleb. Energy and delay aware task assignment mechanism for UAV-based IoT platform. *IEEE Internet Things J.* **2019**, 6, 3332–3344.
18. A. Hayajneh, M. Hafeez, S. A. R. Zaidi, and D. McLernon. TinyML Empowered Transfer Learning on the Edge. *IEEE Open Journal of the Communications Society*, NaN. [Online]. Available: <https://doi.org/10.1109/OJCOMS.2024.3373177>
19. N. Lane et al.. DeepX: A software accelerator for low-power deep learning inference on mobile devices. in *Proc. 15th Int. Conf. Inf. Process. Sensor Networks (IPSN)*, 2016, pp. 1–12.
20. H. Ren, D. Anicic, and T. A. Runkler. TinyOL: TinyML with Online-Learning on Microcontrollers. *None*, 2021. [Online]. Available: <https://doi.org/10.1109/ijcnn52387.2021.9533927>

21. N. Schizas, A. Karras, C. Karras, and S. Sioutas. TinyML for Ultra-Low Power AI and Large Scale IoT Deployments: A Systematic Review. Multidisciplinary Digital Publishing Institute, 2022. [Online]. Available: <https://doi.org/10.3390/fi14120363>
22. R. Shokri and V. Shmatikov. Privacy-preserving deep learning. in Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), 2015, pp. 1310–1321.
23. Pavan, M., Caltabiano, A., & Roveri, M. (2022). On-device Subject Recognition in UWB-radar Data with Tiny Machine Learning. En R. Lazcano Lopez, D. Madroñal Quintin, F. Palumbo, C. Pilato, & A. Tacchella (Eds.), Proceedings of the CPS Summer School PhD Workshop 2022 (Vol. 3252). CEUR-WS.org. urn:nbn:de:0074-3252-7. [Online]. Available: <https://ceur-ws.org/Vol-3252/paper5.pdf>
24. A. Yousefpour, G. Ishigaki, and J. P. Jue. Fog computing: Towards minimizing delay in the Internet of Things. in Proc. IEEE Int. Conf. Edge Comput. (EDGE), 2017, pp. 17–24.
25. B. Sudharsan et al.. TinyML Benchmark: Executing Fully Connected Neural Networks on Commodity Microcontrollers. None, 2021. [Online]. Available: <https://doi.org/10.1109/wf-iot51360.2021.9595024>
26. Adafruit, "Adafruit EdgeBadge - TensorFlow Lite for Microcontrollers," [Online]. Available: <https://www.adafruit.com/product/4400>. [Accessed: Jan. 27, 2025].
27. Arducam, "Pico4ML-BLE TinyML Dev Kit," [Online]. Available: <https://www.arducam.com/downloads/B0330-Pico4ML-BLE-User-Manual.pdf>. [Accessed: Jan. 27, 2025].
28. Arduino, "Arduino Nano 33 BLE Sense," [Online]. Available: <https://docs.arduino.cc/resources/datasheets/ABX00031-datasheet.pdf>. [Accessed: Jan. 27, 2025].
29. STMicroelectronics, "B-L475E-IOT01A - STM32L4 Discovery kit IoT node," [Online]. Available: <https://www.st.com/en/evaluation-tools/b-l475e-iot01a.html>. [Accessed: Jan. 27, 2025].
30. Espressif Systems, "ESP32-S3-DevKitC-1," [Online]. Available: <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html>. [Accessed: Jan. 27, 2025].
31. Himax, "Himax WE-I Plus EVB Endpoint AI Development Board," [Online]. Available: <https://www.sparkfun.com/himax-we-i-plus-evb-endpoint-ai-development-board.html>. [Accessed: Jan. 27, 2025].
32. NVIDIA, "Jetson Nano Developer Kit," [Online]. Available: <https://developer.nvidia.com/embedded/downloads>. [Accessed: Jan. 27, 2025].
33. Arduino, "Portenta H7," [Online]. Available: <https://docs.arduino.cc/resources/datasheets/ABX00042-ABX00045-ABX00046-datasheet.pdf>. [Accessed: Jan. 27, 2025].
34. Raspberry Pi, "Raspberry Pi 4 Model B," [Online]. Available: <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf>. [Accessed: Jan. 27, 2025].
35. Raspberry Pi, "Raspberry Pi Pico Datasheet," [Online]. Available: <https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf>. [Accessed: Jan. 27, 2025].
36. Seeed Studio, "Seeeduino XIAO," [Online]. Available: <https://wiki.seeedstudio.com/Seeeduino-XIAO/>. [Accessed: Jan. 27, 2025].
37. Sony, "Spresense Products," [Online]. Available: <https://developer.sony.com/spresense/products>. [Accessed: Jan. 27, 2025].
38. SparkFun, "SparkFun Edge Hookup Guide," [Online]. Available: <https://learn.sparkfun.com/tutorials/sparkfun-edge-hookup-guide/all>. [Accessed: Jan. 27, 2025].
39. Syntiant Corp., "Syntiant TinyML," [Online]. Available: <https://www.digikey.com/en/products/detail/syntiant-corp/SYNTIANT-TINYML/15293343>. [Accessed: Jan. 27, 2025].
40. Seeed Studio, "Get Started with Wio Terminal," [Online]. Available: <https://wiki.seeedstudio.com/Wio-Terminal-Getting-Started/>. [Accessed: Jan. 27, 2025].
41. Y. Abadade, N. Benamar, M. Bagaa, and H. Chaoui. Empowering Healthcare: TinyML for Precise Lung Disease Classification. Future Internet, 2024. [Online]. Available: <https://doi.org/10.3390/fi16110391>
42. Y. Chen, T. Krishna, J. Emer, and V. Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. IEEE J. Solid-State Circuits **2017**, 52, 127–138.

43. L. Bai, Y. Zhao and X. Huang, "A CNN Accelerator on FPGA Using Depthwise Separable Convolution," in *IEEE Transactions on Circuits and Systems II: Express Briefs* 65, 1415-1419, Oct. 2018, doi: 10.1109/TCSII.2018.2865896
44. J. Koomey. Growth in data center electricity use 2005 to 2010. Analytics Press, 2011.
45. F. Dehrouyeh, L. Yang, F. B. Ajaei, and A. Shami. On TinyML and Cybersecurity: Electric Vehicle Charging Infrastructure Use Case. *IEEE Access*, 2024. [Online]. Available: <https://doi.org/10.1109/ACCESS.2024.3437192>
46. S. A. R. Zaidi, A. M. Hayajneh, M. Hafeez, and Q. Z. Ahmed. Unlocking Edge Intelligence Through Tiny Machine Learning (TinyML). Institute of Electrical and Electronics Engineers, 2021. [Online]. Available: <https://doi.org/10.1109/access.2022.3207200>
47. V. J. Reddi et al.. Widening Access to Applied Machine Learning with TinyML. None, 2022. [Online]. Available: <https://doi.org/10.1162/99608f92.762d171a>
48. J. Conde, A. Munoz-Arcentales, L. Alonso, J. Salvacha, and G. Huecas. Enhanced FIWARE-Based Architecture for Cyberphysical Systems With Tiny Machine Learning and Machine Learning Operations: A Case Study on Urban Mobility Systems. None, 2024. [Online]. Available: <https://doi.org/10.1109/MITP.2024.3421968>
49. F. Chollet. Xception: Deep learning with depthwise separable convolutions. in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1251–1258.
50. L. Capogrosso, F. Cunico, D. S. Cheng, F. Fummi, and M. Cristani. A Machine Learning-Oriented Survey on Tiny Machine Learning. *IEEE Access*, 2023. [Online]. Available: <https://doi.org/10.1109/access.2024.3365349>
51. R. Alaa, E. Hussein, and H. Al-libawy. OBJECT DETECTION ALGORITHMS IMPLEMENTATION ON EMBEDDED DEVICES: CHALLENGES AND SUGGESTED SOLUTIONS. *Kufa journal of Engineering*, 2024. [Online]. Available: <https://doi.org/10.30572/2018/kje/150309>
52. Y. Abadade, T. Anas, H. Bamoumen, N. Benamar, Y. Chtouki, and A. Hafid. A Comprehensive Survey on TinyML. Institute of Electrical and Electronics Engineers, 2022. [Online]. Available: <https://doi.org/10.1109/access.2023.3294111>
53. P. Wiese et al.. Toward Attention-based TinyML: A Heterogeneous Accelerated Architecture and Automated Deployment Flow. None, 2024. [Online]. Available: <https://doi.org/10.1109/MDAT.2025.3527371>
54. A. Datta et al.. Real-Time Air Quality Predictions for Smart Cities using TinyML. in *International Conference of Distributed Computing and Networking*, 2024. [Online]. Available: <https://doi.org/10.1145/3631461.3631947>
55. Y. Zhang, D. Wijerathne, Z. Li, and T. Mitra. Power-Performance Characterization of TinyML Systems. in *ICCD*, 2022. [Online]. Available: <https://doi.org/10.1109/ICCD56317.2022.00099>
56. M. Rb, P. Tüchel, A. Sikora, and D. Mueller-Gritschneider. A Continual and Incremental Learning Approach for TinyML On-device Training Using Dataset Distillation and Model Size Adaption. in *Industrial Cyber-Physical Systems*, 2024. [Online]. Available: <https://doi.org/10.1109/ICPS59941.2024.10639989>
57. A. Karras et al.. TinyML Algorithms for Big Data Management in Large-Scale IoT Systems. Multidisciplinary Digital Publishing Institute, 2024. [Online]. Available: <https://doi.org/10.3390/fi16020042>
58. J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, and S. Han. Tiny Machine Learning: Progress and Futures [Feature]. Institute of Electrical and Electronics Engineers, 2022. [Online]. Available: <https://doi.org/10.1109/mcas.2023.3302182>
59. T. Chen et al.. TVM: End-to-end optimization stack for deep learning. in *Proc. 13th USENIX Symp. Operating Syst. Design Implementation (OSDI)*, 2018, pp. 1–15.
60. H. Oufettoul, R. Chaibi, and S. Motahhir. TinyML Applications, Research Challenges, and Future Research Directions. None, 2024. [Online]. Available: <https://doi.org/10.1109/LT60077.2024.10469633>
61. J. Zhang and J. Li. Improving the performance of openCL-based FPGA accelerator for convolutional neural network. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* **2019**, 27, 2783–2794.

62. S. Jaiswal, R. Goli, A. Kumar, V. Seshadri, and R. Sharma. MinUn: Accurate ML Inference on Microcontrollers. in ACM SIGPLAN Conference on Languages, Compilers, and Tools for Embedded Systems, 2022. [Online]. Available: <https://doi.org/10.1145/3589610.3596278>
63. K. Ovtcharov et al.. Accelerating deep convolutional neural networks using specialized hardware. Microsoft Research Whitepaper, 2015
64. Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief. A survey on mobile edge computing: The communication perspective. *IEEE Commun. Surv. Tuts.* **2017**, 19, 2322–2358.
65. K. Kim, S. Jang, J.-H. Park, E. Lee, and S.-S. Lee. Lightweight and Energy-Efficient Deep Learning Accelerator for Real-Time Object Detection on Edge Devices. Multidisciplinary Digital Publishing Institute, 2023. [Online]. Available: <https://doi.org/10.3390/s23031185>
66. V. Tsoukas, A. Gkogkidis, E. Boumpa, and A. Kakarountas. A Review on the Emerging Technology of TinyML. *ACM Computing Surveys*, 2024. [Online]. Available: <https://doi.org/10.1145/3661820>
67. R. G. Gallager. A perspective on multiaccess channels. *IEEE Trans. Inf. Theory* **1985**, 31, 124–142.
68. A. Costan and S. Devadas. Intel SGX explained. IACR Cryptology ePrint Archive, 2016.
69. S. King and S. Nadal. PPCoin: Peer-to-peer crypto-currency with proof-of-stake. Self-Published Paper, 2012.
70. M. Al Faruque and L. V. Mancini. Energy management-as-a-service over fog computing platform. *IEEE Internet Things J.* **2016**, 3, 161–169.
71. B. Sudharsan et al.. TinyML Benchmark: Executing Fully Connected Neural Networks on Commodity Microcontrollers. None, 2021. [Online]. Available: <https://doi.org/10.1109/wf-iot51360.2021.9595024>
72. N. Andalib and M. Selimi. Exploring Local and Cloud-Based Training Use Cases for Embedded Machine Learning. in Mediterranean Conference on Embedded Computing, 2024. [Online]. Available: <https://doi.org/10.1109/MECO62516.2024.10577837>
73. M. R. Azimi et al.. Hich: Hierarchical fog-assisted computing architecture for healthcare IoT. *ACM Trans. Embed. Comput. Syst.* **2020**, 19, 1–29.
74. C. H. Liu et al.. Intelligent edge computing for IoT-based energy management in smart cities. *IEEE Netw.* **2019**, 33, 111–117.
75. K. Xu et al.. An Ultra-Low Power TinyML System for Real-Time Visual Processing at Edge. Institute of Electrical and Electronics Engineers, 2023. [Online]. Available: <https://doi.org/10.1109/tcsii.2023.3239044>
76. T. Li, A. K. Sahu, A. Talwalkar, and V. Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Process. Mag.* **2020**, 37, 50–60.
77. P. Mach and Z. Becvar. Mobile edge computing: A survey on architecture and computation offloading. *IEEE Commun. Surv. Tuts.* **2017**, 19, 1628–1656.
78. V. Tsoukas, E. Boumpa, G. Giannakas, and A. Kakarountas. A Review of Machine Learning and TinyML in Healthcare. in Panhellenic Conference on Informatics, 2021. [Online]. Available: <https://doi.org/10.1145/3503823.3503836>
79. M. Neethirajan. Recent advances in wearable sensors for animal health management. *Sens. Bio-Sens. Res.* **2017**, 12, 15–29.
80. L. T. Yang et al.. A survey on smart agriculture: Development modes, technologies, and security and privacy challenges. *IEEE/CAA J. Autom. Sinica* **2021**, 8, 273–302.
81. S. Yin, X. Li, and H. Gao. Data-based techniques focused on modern industry: An overview. *IEEE Trans. Ind. Electron.* **2015**, 62, 657–667.
82. P. Arpaia et al.. Accurate Energy Measurements for Tiny Machine Learning Workloads. None, 2024. [Online]. Available: <https://doi.org/10.1109/MetroXRAINE62247.2024.10797006>
83. K. Crawford. The hidden biases in big data. *Harv. Bus. Rev.*, Apr. 2013.
84. L. T. Yang et al.. NetAdapt: Platform-aware neural network adaptation for mobile applications. in Proc. Eur. Conf. Comput. Vis. (ECCV), 2018, pp. 285–300.
85. M. Horowitz. Computing's energy problem (and what we can do about it). in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Pap. (ISSCC)*, 2014, pp. 10–14.
86. C. Gentry. Fully homomorphic encryption using ideal lattices. in *Proc. 41st Annu. ACM Symp. Theory Comput.*, 2009, pp. 169–178.

87. G. Signoretti et al.. An Evolving TinyML Compression Algorithm for IoT Environments Based on Data Eccentricity. Multidisciplinary Digital Publishing Institute, 2021. [Online]. Available: <https://doi.org/10.3390/s21124153>
88. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. C. Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 4510–4520.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.